Kernel-Checked Bridges from OpenAI Math 378 to Two Formal ETH Predicates

Formalize.now Technical note, version 3

November 3, 2026

Abstract

We document two kernel-checked Lean 4.34.1 bridges. The first derives the negation of OpenAI ParityLifts' `PositiveRateETH` from OpenAI's formal bounded-width SAT theorem, by exact input conversion, machine simulation, and asymptotic rate transfer. The second proves the independent implication ¬PositiveRateETH ⇒ MainResults.ETHIsFalse, connecting this predicate to Anthropic's ETH formalization by a verified ParityLifts-to-Mathlib `FinTM2` simulation and a polynomial-time input encoding converter. The two final theorems were built on a VPS (11,600 and 8,985 jobs respectively); the cross-formalization passed Comparator's Lean default kernel check. Both dependency audits report only `propext`, `Classical.choice`, and `Quot.sound`. The new theorem establishes an implication between two exact formal predicates, not an equivalence with textbook ETH.

1 Statement of the result

The final declaration in `OAI/Bridges/ETH/FinalImplication.lean` is:

```
theorem not_positiveRateETH :
¬ OAI.ParityWL.Complexity.PositiveRateETH :=
not_positiveRateETH_of_mainTheorem
OAI.Algorithms.SubexponentialSAT.SqrtLogSpace.original_sat_main
```

The target predicate is defined using a deterministic finite-alphabet multitape machine M, CNF instances $F : \mathrm{Formula}(N)$ whose clauses have at most three literals (repeated clauses allowed), and a worst-case execution-time relation SATTimeBound:

$$\mathrm{PositiveRateETH} := \exists \delta > 0,\ \neg\exists M, C, d\ (C > 0 \wedge \mathrm{SATTimeBound}(M,\ (N, L) \mapsto C(L+1)^d 2^{\delta N})). \quad (1)$$

Here N denotes the declared variable count and L the binary encoding length. Thus its negation supplies, for each $\delta > 0$, an appropriate fixed machine and polynomial factor valid uniformly over inputs. The quantifier order is the positive-rate form associated with $s_3 > 0$ in Impagliazzo–Paturi [5]. To see why, let $A(\delta)$ assert that a uniform deterministic decider with some polynomial input-length factor meets the $2^{\delta N}$ rate. For $0 < \delta_1 \le \delta_2$, $A(\delta_1)$ implies $A(\delta_2)$, using the same machine and constants. Hence an unattainable positive rate is accompanied by unattainability at every smaller positive rate. Negating Eq. (1) gives $\forall \delta > 0,\ A(\delta)$: the machine may depend on δ, while each selected machine works on all encoded instances. The interpretation of the specific machine and encoding conventions is addressed in Section 6.

2 Source theorem and complexity bound

OpenAI's source theorem, `SqrtLogSpace.original_sat_main`, has type `SqrtLog.Statement.MainTheorem` [1]. For every fixed clause width $k \ge 3$, `Feasible k` holds. The latter asserts a primitive

multitape machine deciding satisfiability of width-k CNF inputs under OpenAI's canonical encoding, on every such input, with no promise or large-input qualification, and with positive constants C, A and positive degree d that depend on k only.

The Lean-verified runtime uses

$$T(n, L) \le C(L+1)^d 2^{A\,\rho(n)}, \qquad \rho(n) = \frac{n\,[\log\log(n+16)]^2}{\sqrt{\log(n+16)}}. \tag{2}$$

where n is the variable count of the OpenAI instance and L is the length of its encoding `Encoding.encodeCNF`. This is subexponential in n up to the polynomial input-length factor since $\rho(n)/n \to 0$. The associated manuscript discusses a sharper exponent with one rather than two powers of $\log\log n$; the bridge uses the Lean theorem in Eq. (2).

3 Construction of the bridge

3.1 Exact encoding conversion

The ParityLifts formula representation allows lists of clauses over `Fin N`, each with at most three literals. The conversion constructs a canonical CNF word and establishes both (i) satisfiability preservation and (ii) bit-for-bit equality with OpenAI's `Encoding.encodeCNF` (`canonicalWord_eq`). The executable `canonicalWordProgram` runs on the normalized bit form `minimizeList F.bitList` that ParityLifts' pipeline supplies, and `canonicalWordProgram_time_poly` bounds it by

$$\exists c, d \in \mathbb{N}\ \forall N\ \forall F : \mathrm{Formula}(N), \quad \mathrm{time}_{\mathrm{conv}}\big(\mathrm{minimizeList}(F.\mathrm{bitList})\big) \le c(N+1)^d, \tag{3}$$

where $\mathrm{time}_{\mathrm{conv}}$ is the step count of `canonicalWordProgram`. The separate `ConverterInterface` module proves `no_polyProgram_canonicalWord`: no `PolyProgram`, whose cost is polynomial in the serialized input size alone, computes `canonicalWord`, because variable names written in binary become values whose output length grows with their magnitude. The bridge therefore uses a `TreeProgram` whose time is polynomial in the declared variable count N, which the target rate $2^{\delta N}$ absorbs. This module lies outside the final theorem's import closure.

3.2 Machine translation and executable simulation

`MachineTranslation.lean` represents source tape cells `none`, `some false`, and `some true` by three symbols, reindexes tapes, and proves a correspondence of initial configurations and individual transition steps. A source-machine halting trace within s steps produces an output trace in the target machine model (`emits_of_outputsInTime`).

`MachineRunner.lean` builds a single target program `primitiveRunner M` that probes increasing step budgets, distinguishes a completed false answer from a still-running computation, and returns the eventual Boolean answer. `MachineRunnerCost.lean` then establishes

$$\begin{aligned} \forall M\ \exists c, e \in \mathbb{N}\ \forall w, b, s, \quad & M.\mathrm{OutputsInTime}(w, b, s) \Longrightarrow \\ & \mathrm{RBEval}\big(\mathrm{primitiveRunner}(M), \mathrm{treeEncode}(w), \\ & \quad c(s + |w| + 1)^e, \mathrm{treeEncode}(b)\big). \end{aligned} \tag{4}$$

This is `primitiveRunner_poly`, stated as `PrimitiveSimulation M (primitiveRunner M) c e`. The constants c, e depend on M only. The declaration `simulationHypothesis_proved` closes the previously conditional simulation interface.

3.3 Exponential-rate transfer

The rate argument isolates two effects. The conversion and simulation introduce polynomial factors in N and the source running time. Separately, OpenAI's exponential factor $2^{A\rho(N)}$ satisfies, for every $r > 0$, a bound of the form $K(N+1)^D 2^{rN}$, by `runtimeFactor_expBound`; this also covers the e-th power introduced by the simulation, since $(2^{A\rho(N)})^e = 2^{eA\rho(N)}$. The composition and SAT decoding lemmas combine the bounds at any chosen positive rate δ, producing the negation of Eq. (1). The machine is constructed independently of the instance and its size. The verified final theorem uses the precise existential quantifiers stated in Eq. (1).

4 Anthropic ETH cross-formalization (RPI-ETH-010)

The second bridge establishes a direct implication between the *formal* ETH predicates defined in the two projects. In the pinned Lean environment its final declaration is:

```
theorem OAI.Bridges.AnthropicETH.
not_positiveRateETH_implies_ethIsFalse :
¬ OAI.ParityWL.Complexity.PositiveRateETH →
MainResults.ETHIsFalse
```

The theorem is unconditional: its earlier `SimulationHypothesis` and `ConverterHypothesis` premises were discharged by the checked declarations `simulationHypothesis_proved` and `converterHypothesis_proved`. In particular, it is not the trivial implication using Anthropic's already-existing proof of `ETHIsFalse`. A transitive composition with the original bridge supplies a proof of Anthropic's formal target, while the cross-formalization theorem itself has a proof closure independent of both existing ETH/SAT proofs.

4.1 Machine semantics and input conversion

The source machine is ParityLifts' deterministic finite-alphabet multitape machine; the target is Mathlib's stack-based `FinTM2` model, with an explicit initial configuration and a halting convention that empties work stacks and resets the state. The simulation represents each tape by left/right stacks, establishes one-step correspondence, loads the input and cleans up the result. For a source emission within s steps on word x, the proved target bound is

$$T_{\mathrm{FinTM2}}(x, s) \le 4(|x| + s + 1). \tag{5}$$

The conversion goes in the direction needed for the implication: Anthropic's `encClauses 3` encoding of a 3-CNF instance is transformed into ParityLifts' `encodeFormula` of the corresponding formula. On supported formulas, `Conv.conv_encClauses` proves exact output equality, and the satisfiability translation is proved separately. The converter is an `FCode/PolyProgram` with polynomial running time on *all* inputs, not only well-formed encodings. Its composition overhead has a degree fixed independently of the downstream SAT machine.

4.2 Rates, quantifiers, and dependencies

Anthropic's input instances have no repeated clauses. Their count and encoded length are polynomially bounded in the number n of variables, so the polynomial-length overhead can be absorbed into $2^{\varepsilon n}$ at

an arbitrarily small additional positive rate. The bridge chooses a source rate small enough to account for the fixed preprocessing degree and then derives Anthropic's `InTime 3` δ at each positive δ; the infimum-rate definition gives `MainResults.ETHIsFalse`. A different source machine may be chosen at each rate, matching the quantifiers of both negated predicates.

The source audit rejects forbidden module imports, out-of-scope OpenAI constants, and unwanted axioms in the final theorem's dependency closure. The copied Anthropic `MainResults.lean` has SHA-256 `47f78a9550d95b37d8718dd1ee16ae824da893a5e17c319f737e6d0a5ed24944`. The final theorem's reachable-constant audit reports 19,867 constants with only the three standard axioms. The source theorem and the independent check are available in the cross-formalization branch and its VPS report.

5 Verification and reproducibility

Item	Recorded value
Upstream revision	`7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`
Verified C2 snapshot	`81ec97cd23d21675cee4ee76768fffba33a61e17`
Documentation C3 commit	`81708f93f` (VPS abbreviated hash)
C2 tree hash	`1637f35b64e7f9f5d8f3c4e44e28007812a74f72`
Toolchain	Lean 4.34.1; pinned Mathlib manifest
Build target	`OAI.Bridges.ETH.FinalImplication`
Independent VPS result	11,600 jobs; exit status 0
Final axiom dependencies	`propext`; `Classical.choice`; `Quot.sound`

To reproduce from the pinned project under `lean/`, run:

```
LEAN_NUM_THREADS=2 lake build OAI.Bridges.ETH.FinalImplication
```

The module includes `#print axioms OAI.Bridges.ETH.not_positiveRateETH`. Its axiom audit tests the complete proof dependencies, including the OpenAI theorem imported as a proof term. The original OpenAI sources and Comparator settings remain unchanged by the bridge. The companion reproducibility guide [2] specifies the other modules, a source-integrity scan, and optional Comparator checks. OpenAI's Comparator configuration checks `exact_count_and_sat`; `original_sat_main` is derived from it within OpenAI's sources and is checked by the Lean kernel during this build. Commit hashes differ between the authoring environment and the VPS because patches were applied with `git am`; the tree hash is the environment-independent identifier.

For RPI-ETH-010, the separate VPS run on proof commit `d6b79b7d1` built `OAI.Bridges.AnthropicETH.Audit` successfully (8,985 jobs) with Lean 4.34.1 and pinned Mathlib. The audit checked the final theorem, including its reachable-constant guard and axiom restrictions. The Comparator challenge `AnthropicETHCrossFormalization` passed with exit zero and the messages "Lean default kernel accepts the solution" and "Your solution is okay!" The VPS Comparator configuration disabled nanoda; a *separate local* standalone nanoda replay accepted the export. These are distinct checks and should not be conflated. Reproduction instructions and tool revisions are recorded in the RPI-378.3d report. The registry entry is RPI-ETH-010.

6 Formal scope and related work

The conclusion's meaning is determined by `PositiveRateETH` and its dependencies: `SATTimeBound`, machine configurations and transitions, `Emits`, the definition of 3-CNF formulas, and `encodeFormula`.

OpenAI's `MainTheorem` enters as an already kernel-checked proof term; it is absent from the final theorem's type. In our reading of these definitions, the predicate states the $s_3 > 0$ deterministic 3-SAT formulation [5] with the same quantifier order, and each of its conventions asks at least as much of the refuting algorithm as the standard one: a deterministic finite-alphabet multitape Turing machine, worst-case step counts to an absorbing output state, clauses of width at most three, and an exponent measured in the declared variable count N with a polynomial factor in the encoding length. The monotonicity of admissible exponential rates explains the infimum-based formulation. Equivalence across machine models and specific standard input encodings remains a mathematical interpretation of the formal specifications, separate from the kernel check.

Anthropic's *formal-math/eth* [3] has a separately defined `ETHIsFalse` target and a different machine model and toolchain. Section 4 now establishes the *forward implication* at the Lean theorem level, using a byte-identical copy of Anthropic's trusted definition file without importing its existing ETH proof. An independent cross-formalization bridge was also discussed in Anthropic's public issue tracker [4]. The reverse direction, and especially equivalence to the full range of textbook ETH formulations, are not established by the present proofs.

A further target is the separately formalized `UniformSubexponentialETH`, in which one machine with one polynomial degree must satisfy every positive-rate bound. The existing construction suggests a route to that stronger result, provided the complete decider and its polynomial degree can be fixed before the rate is selected; the theorem documented here is the checked `not_positiveRateETH` declaration.

7 Availability and next checks

The original proof and C3 report are public on the component-B branch of `thinkthoughts/math`. See `ETH_BRIDGE_REPORT.md`, `ETH_BRIDGE_REPRODUCE.md`, and `ETH_BRIDGE_MANIFEST.md`. The new cross-formalization proof and VPS record are on the RPI-378.3 branch and in its verification report. The indexed implication is RPI-ETH-010. Further checks concern semantic fidelity to textbook ETH, independent reproduction with a second checker, and a possible reverse machine-and-encoding reduction.

References

[1] OpenAI, *math*, pinned revision `7ef5c0c`, https://github.com/openai/math.

[2] *ETH bridge verification and reproducibility documentation*, https://github.com/thinkthoughts/math/tree/eth-bridge/component-B/docs.

[3] Anthropic, *formal-math*, ETH development, https://github.com/anthropics/formal-math/tree/main/eth.

[4] Anthropic formal-math issue tracker, cross-formalization discussion, https://github.com/anthropics/formal-math/issues/36.

[5] R. Impagliazzo and R. Paturi, *On the Complexity of k-SAT*, Journal of Computer and System Sciences 62 (2001), 367–375.